

LoveLive! 综合管理系统 — 架构设计文档

作者：王玺玮 学号：PB23071333 日期：2026-06-01

目录

- [1. 项目概述](#)
- [2. 需求分析](#)
- [3. 系统架构设计](#)
- [4. ER 图设计](#)
- [5. 数据库表详细设计](#)
- [6. 功能模块设计](#)
- [7. 前端页面设计](#)
- [8. 技术选型](#)
- [9. 部署与公开方案](#)
- [10. 后期预期进展](#)

1. 项目概述

1.1 项目背景

LoveLive!（ラブライブ！）是由日本动画公司 SUNRISE、唱片公司 Lantis 及月刊杂志电击 G's Magazine 联合推出的跨媒体偶像企划。自 2010 年启动以来，已衍生出 μ's、Aqours、虹咲学园学园偶像同好会、Liella! 等多个偶像团体，在全球拥有大量粉丝。

粉丝群体中，“舞团”（舞蹈翻跳团体）是一个重要的线下活动形式。舞团成员（舞见）会模仿动漫角色的服装和舞蹈，进行舞台排练与表演。然而目前缺乏一个能够将**角色信息查询**与**舞团排练时间管理**结合起来的一站式平台。

1.2 项目目标

构建一个 LoveLive! 综合管理系统，包含两个核心子系统：

子系统	核心功能
动漫角色介绍子系统	展示企划、动漫团、角色、声优等详细信息，支持多维度筛选查询
舞台排练时间收集子系统	管理舞团排练日程，支持增删改查，并与角色系统联动

两个子系统通过**动漫团—舞团**、**角色—排练参与**的关系有机串联，实现数据互通与页面跳转。

1.3 系统范围

- 数据规模预估**：约 5 个企划、10 个动漫团、100+ 角色、50+ 声优、若干舞团及排练记录
- 用户群体**：LoveLive 粉丝、舞团组织者、舞见
- 平台**：Web 应用

2. 需求分析

2.1 动漫角色介绍子系统

2.1.1 角色信息管理

每个角色需至少包含：

- **角色名**（如：高坂穗乃果、绚濑绘里）
- **角色图片**（多模态信息，支持 URL 或本地存储路径）
- **所属动漫团**（如 μ's、Aqours）
- **声优信息**（声优姓名、出生日期/年龄、性别）
- **扩展信息**：生日、血型、身高、爱好、角色简介

2.1.2 动漫团信息管理

每个动漫团需包含：

- **团名**（如 μ's、Aqours、虹咲学园学园偶像同好会、Liella!）
- **所属企划**（如 LoveLive!、LoveLive! Sunshine!!）
- **成员列表**（关联到角色表）
- **团的介绍**（可多段落：代表作、历史沿革、风格特点等）
- **成立时间 / 解散时间**
- **团徽/合照图片**

2.1.3 声优信息管理

- 声优姓名
- 出生日期（系统自动计算年龄）
- 性别
- 所属事务所

2.1.4 企划信息管理

- 企划名称
- 企划简介
- 起止时间

2.1.5 查询与筛选

要求支持以下筛选维度：

- 按**角色名**模糊搜索
- 按**动漫团名**筛选
- 按**企划名**筛选
- 按**女声优年龄范围**筛选（如"25-35岁"）
- 组合条件筛选

2.2 舞台排练时间收集子系统

2.2.1 排练信息管理

每条排练记录包含：

- 舞团名称
- 舞见 CN（Cosplayer Name，参与排练的舞者）
- 排练日期
- 排练时间段（开始时间 ~ 结束时间）
- 排练地点
- 人数上限（该时段最多容纳多少人）
- 当前参与人数（系统自动统计已参与的舞见数，直观展示饱和度）
- 参与角色（关联到角色子系统的具体角色）
- 排练内容简介

2.2.2 功能需求

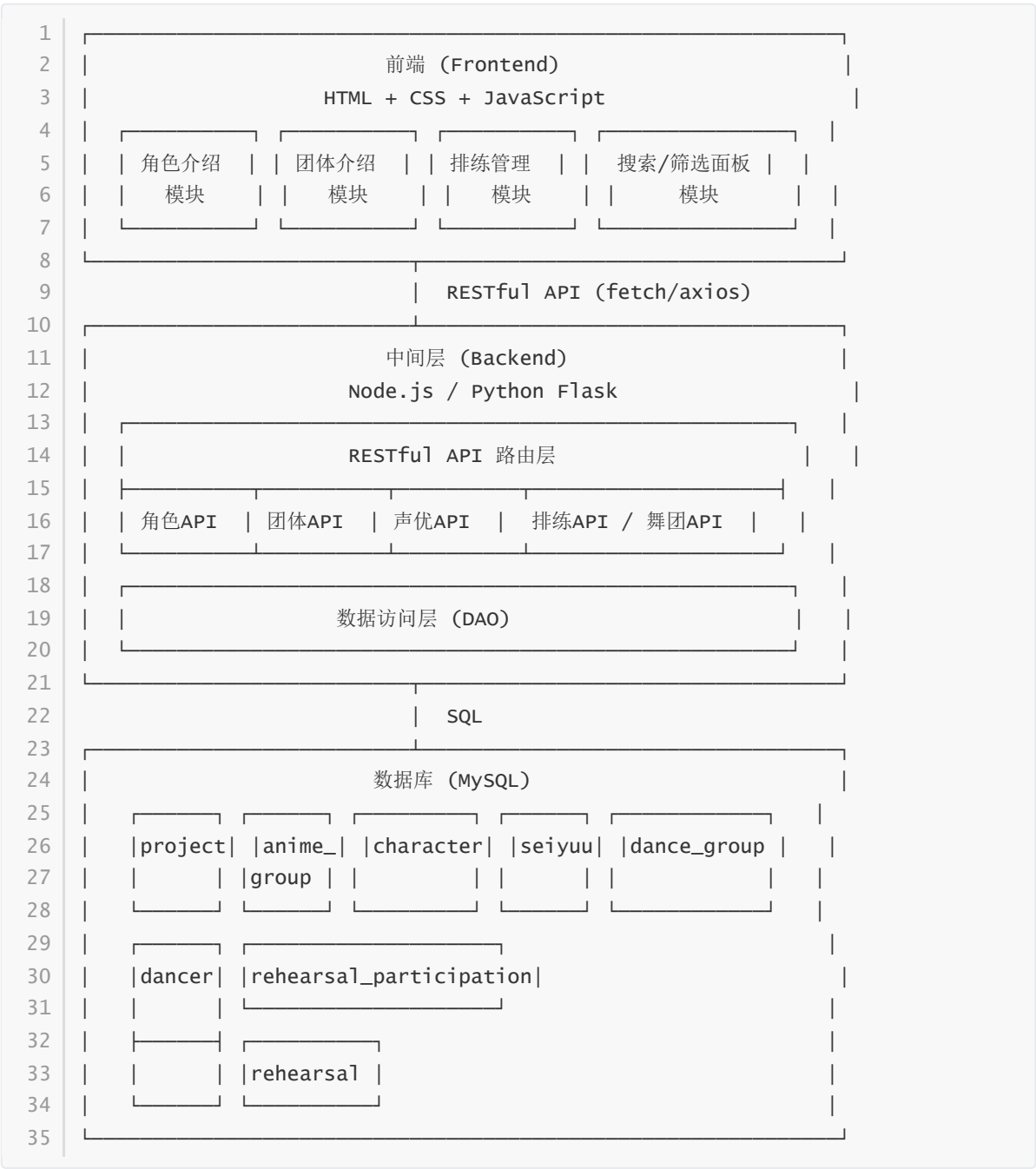
功能	说明
排练时间收集	新增排练记录，含参与角色、舞见信息、人数上限
排练时间查询	按日期、舞团、角色、人数饱和度等条件查询排练安排
时段人数统计	核心功能： 自动统计每个排练时段的参与人数，在日历视图中直观展示饱和度（颜色标识 + 进度条），方便组织者一眼看出哪些时段还有空位、哪些时段已满
排练时间修改	修改已有排练记录的各项字段（含人数上限、参与人员）
排练时间删除	删除排练记录（含确认机制）
人数饱和度筛选	可按"已满 / 即将满 / 有空位"筛选排练时段，快速找到可用时间

2.2.3 跨子系统联动

- 从[动漫团详情页](#)可点击查看排练该团曲目的舞团及排练时间
 - 从[角色详情页](#)可查看有哪些舞见在排练中扮演该角色
 - 从[排练详情](#)可点击跳转到参与角色的介绍页
-

3. 系统架构设计

3.1 总体架构



3.2 架构说明

采用**前后端分离**的三层架构：

层级	技术	职责
表现层	HTML + CSS + JS	页面渲染、用户交互、数据展示
业务逻辑层	Node.js (Express) 或 Python (Flask)	API 接口、业务逻辑、数据校验
数据层	MySQL	数据持久化、关系管理、查询优化

3.3 子系统间关联设计

两个子系统通过以下两条主线关联：

- 1. **动漫团** → **舞团**：一个舞团主要翻跳某个动漫团的曲目，因此 `dance_group` 表通过 `anime_group_id` 外键关联到 `anime_group`
- 2. **角色** → **排练参与**：`rehearsal_participation` 表同时关联 `rehearsal`、`dancer` 和 `character`，实现"某舞见在某排练中扮演某角色"的三元关系

4. ER 图设计

4.1 实体关系图 (Mermaid)

4.2 实体说明

实体	中文名	说明
PROJECT	企划	LoveLive 系列的总企划（无印、Sunshine、Superstar 等）
ANIME_GROUP	动漫团	企划内的偶像团体（μ's、Aqours 等）
SEIYUU	声优	为角色配音的声优
CHARACTER	角色	动漫中的偶像角色
DANCE_GROUP	舞团	现实中翻跳 LoveLive 舞蹈的团体
DANCER	舞见	舞团中的成员（Cosplayer）
REHEARSAL	排练	一次排练安排
REHEARSAL_PARTICIPATION	排练参与	记录某舞见在某排练中扮演某角色的三元关联

4.3 关系说明

1	Project —1:N→ Anime_Group	一个企划包含多个动漫团
2	Anime_Group —1:N→ Character	一个动漫团拥有多个角色
3	Seiyuu —1:N→ Character	一个声优配音多个角色
4	Anime_Group —1:N→ Dance_Group	一个动漫团可被多个舞团翻跳
5	Dance_Group —1:N→ Dancer	一个舞团有多个舞见
6	Dance_Group —1:N→ Rehearsal	一个舞团安排多次排练
7	Rehearsal —1:N→ Participation	一次排练有多个参与记录
8	Dancer —1:N→ Participation	一个舞见可参与多次排练
9	Character —1:N→ Participation	一个角色可被多个舞见在不同排练中扮演

5. 数据库表详细设计

5.1 project（企划表）

字段名	类型	约束	说明
project_id	INT	PK, AUTO_INCREMENT	企划唯一标识
name	VARCHAR(100)	NOT NULL, UNIQUE	企划名称，如 "LoveLive!"
description	TEXT		企划简介
start_date	DATE		企划启动日期
end_date	DATE		企划结束日期（NULL 表示仍在进行）

示例数据：

project_id	name	start_date	end_date
1	LoveLive!	2010-06-30	NULL
2	LoveLive! Sunshine!!	2015-04-30	NULL
3	LoveLive! 虹咲学园	2017-09-21	NULL
4	LoveLive! Superstar!!	2020-07-14	NULL

5.2 anime_group（动漫团表）

字段名	类型	约束	说明
group_id	INT	PK, AUTO_INCREMENT	团队唯一标识
project_id	INT	FK → project(project_id), NOT NULL	所属企划
name	VARCHAR(100)	NOT NULL	团名，如 "μ's"、"Aqours"
description	TEXT		团队介绍（支持多段落，用 \n\n 分隔）
representative_works	TEXT		代表作
history	TEXT		历史沿革
founding_date	DATE		成立日期
disband_date	DATE		解散日期（NULL 表示未解散）

字段名	类型	约束	说明
image_url	VARCHAR(500)		团队合影图片 URL

索引: INDEX idx_group_project (project_id), INDEX idx_group_name (name)

5.3 seiyuu (声优表)

字段名	类型	约束	说明
seiyuu_id	INT	PK, AUTO_INCREMENT	声优唯一标识
name	VARCHAR(100)	NOT NULL	声优姓名
birth_date	DATE		出生日期 (用于计算年龄)
gender	ENUM('女','男','其他')	DEFAULT '女'	性别 (大部分 LoveLive 声优为女性)
agency	VARCHAR(100)		所属事务所
image_url	VARCHAR(500)		声优照片 URL

索引: INDEX idx_seiyuu_gender_birth (gender, birth_date) — 用于"按女声优年龄范围筛选"

5.4 character (角色表)

字段名	类型	约束	说明
character_id	INT	PK, AUTO_INCREMENT	角色唯一标识
group_id	INT	FK → anime_group(group_id), NOT NULL	所属动漫团
seiyuu_id	INT	FK → seiyuu(seiyuu_id)	配音声优
name	VARCHAR(100)	NOT NULL	角色名, 如 "高坂穗乃果"
image_url	VARCHAR(500)		角色图片 URL (多模态信息)
description	TEXT		角色简介
birthday	DATE		角色生日
blood_type	ENUM('A','B','AB','O','不明')		血型
height	DECIMAL(5,1)		身高 (cm)
hobby	VARCHAR(500)		爱好

索引: INDEX idx_char_group (group_id), INDEX idx_char_name (name), INDEX idx_char_seiyuu (seiyuu_id)

5.5 dance_group (舞团表)

字段名	类型	约束	说明
dance_group_id	INT	PK, AUTO_INCREMENT	舞团唯一标识
anime_group_id	INT	FK → anime_group(group_id)	主要翻跳的动漫团 (可为 NULL 表示综合舞团)
name	VARCHAR(100)	NOT NULL	舞团名称
description	TEXT		舞团简介
created_date	DATE		舞团创建日期

索引: INDEX idx_dg_anime (anime_group_id)

5.6 dancer (舞见表)

字段名	类型	约束	说明
dancer_id	INT	PK, AUTO_INCREMENT	舞见唯一标识
dance_group_id	INT	FK → dance_group(dance_group_id), NOT NULL	所属舞团
cn_name	VARCHAR(100)	NOT NULL	舞见 CN (Cosplayer Name)
contact_info	VARCHAR(200)		联系方式 (QQ/微信等, 选填)

索引: INDEX idx_dancer_group (dance_group_id)

5.7 rehearsal (排练表)

字段名	类型	约束	说明
rehearsal_id	INT	PK, AUTO_INCREMENT	排练唯一标识
dance_group_id	INT	FK → dance_group(dance_group_id), NOT NULL	所属舞团
rehearsal_date	DATE	NOT NULL	排练日期
start_time	TIME	NOT NULL	排练开始时间
end_time	TIME	NOT NULL	排练结束时间

字段名	类型	约束	说明
location	VARCHAR(200)	NOT NULL	排练地点
max_participants	INT	DEFAULT 0	人数上限 (0 表示不限制; >0 表示该时段最多容纳人数)
content_summary	TEXT		排练内容简介

索引: INDEX idx_reh_date (rehearsal_date), INDEX idx_reh_group (dance_group_id)

人数统计说明: 当前参与人数通过 SELECT COUNT(*) FROM rehearsal_participation WHERE rehearsal_id = ? 实时计算, 也可创建数据库 VIEW 预先 JOIN 以提升查询性能。

5.9 视图: rehearsal_with_count (排练人数统计视图)

为方便前端直接获取每个排练时段的参与人数, 创建数据库视图:

```
1 CREATE VIEW rehearsal_with_count AS
2 SELECT
3     r.*,
4     COUNT(rp.participation_id) AS current_participants,
5     CASE
6         WHEN r.max_participants = 0 THEN 0.0
7         ELSE ROUND(COUNT(rp.participation_id) / r.max_participants * 100, 1)
8     END AS occupancy_rate
9 FROM rehearsal r
10 LEFT JOIN rehearsal_participation rp ON r.rehearsal_id = rp.rehearsal_id
11 GROUP BY r.rehearsal_id;
```

视图字段:

字段名	说明
current_participants	当前已参与的舞见人数
occupancy_rate	饱和度百分比 (0.0~100.0) , max_participants=0 时返回 0.0

5.8 rehearsal_participation (排练参与表)

字段名	类型	约束	说明
participation_id	INT	PK, AUTO_INCREMENT	参与记录唯一标识
rehearsal_id	INT	FK → rehearsal(rehearsal_id), NOT NULL	排练 ID
dancer_id	INT	FK → dancer(dancer_id), NOT NULL	舞见 ID
character_id	INT	FK → character(character_id), NOT NULL	扮演的角色 ID

唯一约束： `UNIQUE (rehearsal_id, dancer_id, character_id)` — 同一排练中同一舞见不能重复扮演同一角色

索引： `INDEX idx_rp_rehearsal (rehearsal_id), INDEX idx_rp_dancer (dancer_id), INDEX idx_rp_character (character_id)`

6. 功能模块设计

6.1 角色介绍子系统

1	动漫角色介绍子系统		
3	企划浏览	团体详情	角色详情
4	- 企划列表	- 团信息	- 角色信息
5	- 所含团体	- 成员列表	- 声优信息
6		- 代表作	- 多模态图片
7		- 历史沿革	
8		- 关联舞团→	
9	声优浏览		
10	- 声优列表	- 年龄筛选	- 所配角色
11			
12			
13			

6.1.1 企划列表页

- 以卡片形式展示各企划
- 点击进入企划详情，展示所含动漫团列表
- 支持企划名模糊搜索

6.1.2 动漫团详情页

- 展示团名、图标、成立/解散日期
- **团介绍：**多段落渲染（代表作、历史沿革、风格特点）
- **成员列表：**头像 + 角色名，点击跳转到角色详情
- **关联舞团：**查看翻跳该团曲目的舞团 → **点击可跳转查看舞团排练时间**
- 筛选功能：按团名搜索

6.1.3 角色详情页

- 角色图片（大图展示，多模态）
- 角色基本信息（姓名、生日、血型、身高、爱好）
- 所属动漫团（可点击跳转）
- 声优信息卡片（姓名、年龄、事务所）→ 可点击跳转声优详情
- **排练记录：**展示有哪些舞团/舞见在排练中扮演了该角色

6.1.4 声优列表页

- 声优列表，支持筛选：
 - 性别筛选（默认可只显示女声优）
 - 年龄范围筛选（如 20-30 岁、30-40 岁等，系统根据 birth_date 自动计算）
- 点击声优查看详情及其配音角色列表

6.2 排练时间收集子系统

1	排练时间收集子系统			
2				
3				
4	舞团管理	排练管理	舞见管理	人数可视化
5	- 新建	- 新增排练	- 添加舞见	- 时段人数统计
6	- 编辑	- 编辑排练	- 管理成员	- 饱和度图表
7	- 关联	- 删除排练		- 满员/空位标识
8		- 查询排练		- 颜色预警
9		- 参与角色		
10		- 人数上限		
11				

6.2.1 排练日历视图（含人数可视化）

- 以日历/周视图形式展示排练安排
- 每个排练条目直观显示：当前人数 / 上限（如 3/5、8/10）
- 颜色饱和度标识：
 - 绿色：饱和度 < 50%（空位充裕，欢迎加入）
 - 黄色：饱和度 50%~80%（即将满员）
 - 橙色：饱和度 80%~99%（仅剩少量空位）
 - 红色：饱和度 ≥ 100%（已满员，无法再加入）
 - 灰色：未设上限（不限制人数）
- 进度条直观展示饱和度
- 可按日期范围筛选
- 可按舞团名称筛选
- 可按参与角色筛选（关联到角色子系统）
- 可按人数饱和度筛选（"仅看有空位" / "仅看已满" / "全部"）
- 点击排练条目查看详情

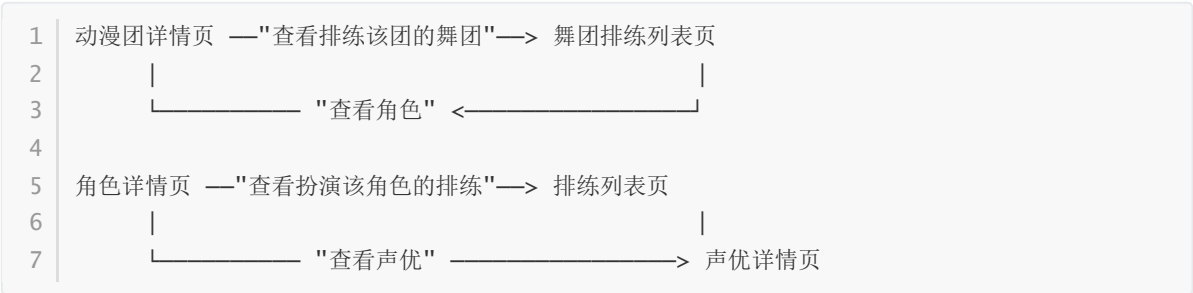
6.2.2 时间冲突与人数预警

- 同一舞团同一时间段自动检测是否有冲突
- 满员排练自动标记，新增参与时若已满给出提示
- 同一舞见同一时间段被安排到不同排练时发出警告

6.2.3 排练 CRUD 操作

操作	说明	要点
新增排练	填写排练表单	舞团、日期、时间段、地点、 人数上限 、内容简介、选择参与角色和舞见
查询排练	多条件组合查询	日期范围 + 舞团 + 角色 + 舞见 + 人数饱和度 等组合筛选
修改排练	编辑已有排练	可修改所有字段，包括调整 人数上限 和参与人员
删除排练	删除排练记录	弹出确认框，同时级联删除关联的参与记录

6.2.3 跨系统跳转设计



7. 前端页面设计

7.1 页面结构



7.2 主要页面

页面	路由	说明
首页	/	仪表盘，展示概览统计
企划列表	/projects	企划卡片网格
企划详情	/projects/:id	企划信息 + 所含动漫团

页面	路由	说明
动漫团详情	/groups/:id	团信息 + 成员 + 关联舞团排练
角色列表	/characters	可筛选的角色表格/卡片
角色详情	/characters/:id	角色完整信息 + 声优 + 排练记录
声优列表	/seiyuus	可筛选年龄的声优列表
舞团列表	/dance-groups	舞团卡片
舞团详情	/dance-groups/:id	舞团信息 + 成员 + 排练日程
排练列表	/rehearsals	排练日历视图 + 筛选面板
排练详情	/rehearsals/:id	排练完整信息 + 参与角色列表

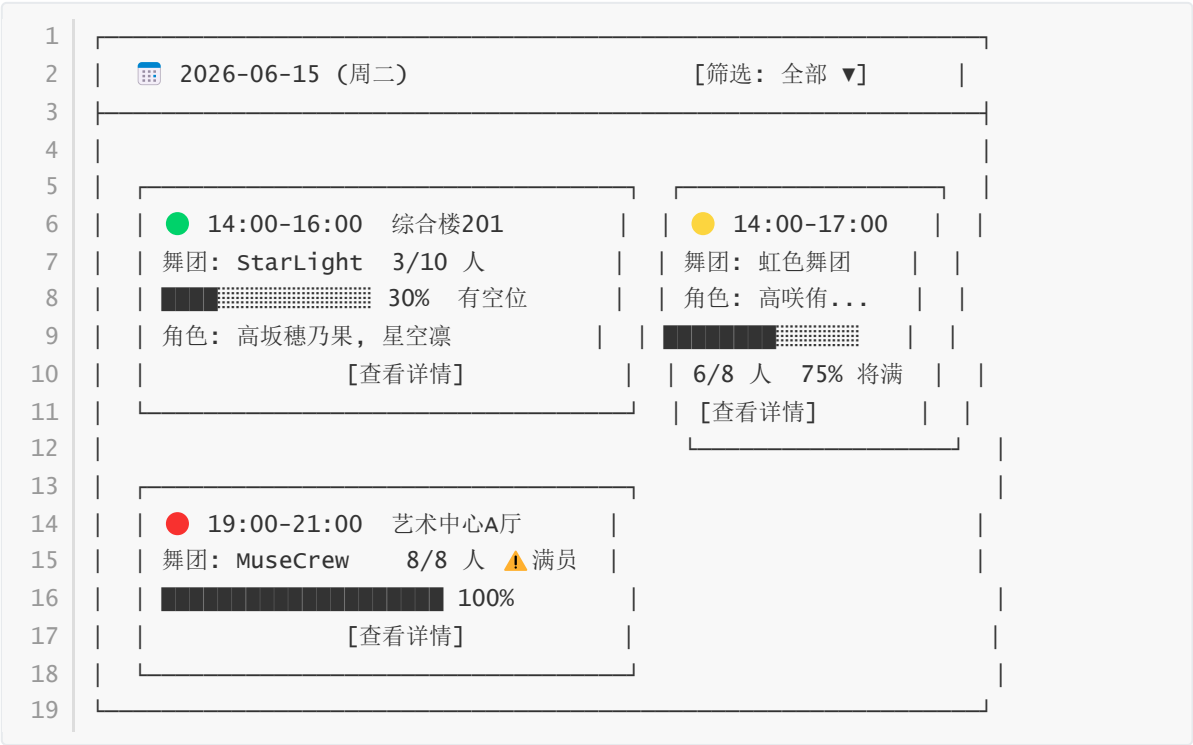
7.3 前端交互特性

- **响应式布局**：适配桌面端和移动端
- **模态框**：新增/编辑排练使用模态弹窗，无需页面跳转
- **面包屑导航**：显示当前页面层级路径
- **图片懒加载**：角色/声优图片按需加载
- **表单验证**：前端校验必填字段（日期、时间、地点等）

7.4 排练人数可视化设计

这是排练系统的核心前端特性，确保时间管理的一目了然。

7.4.1 排练列表视图中的饱和度展示



7.4.2 饱和度颜色与标识规范

饱和度范围	颜色	标签	进度条样式	含义
0% ~ 50%	绿色 #28a745	"有空位"	bg-success	人数充裕，欢迎更多舞见参与
50% ~ 80%	黄色 #ffc107	"将满"	bg-warning	已过半，还有少量位置
80% ~ 99%	橙色 #fd7e14	"即将满员"	bg-orange	仅剩 1-2 个空位
≥ 100%	红色 #dc3545	"已满员" ⚠️ "	bg-danger	无法再加入
上限=0	灰色 #6c757d	"不限人数"	bg-secondary	不设上限

7.4.3 排练详情中的人数面板

1 |

2 | 排练详情

3 |

4 |

参与人数

5 |

6 | 当前：6 人 / 上限：10 人

7 |

60%

8 | 状态：● 将满（还可加入 4 人）

9 |

10 | 参与舞见列表：

11 |

12 |

舞见CN	扮演角色	操作
小明	高坂穗乃果	[移除]
小红	星空凛	[移除]
...	...	...

13 |

14 |

15 |

16 |

17 |

18 |

[+ 添加参与舞见]

19 |

20 |

7.4.4 日历月视图中的人数热力标识

在月视图中，每个日期格内的排练条目用颜色圆点标记饱和度，帮助用户快速定位高负载日期：

1							
2	周一	周二	周三	周四	周五	周六	周日
3	6/9	6/10	6/11	6/12	6/13	6/14	6/15
4							
5			● 14:00		● 16:00	● ●	● 10:00
6			2/5		6/8	两场皆满	2/10
7							

8. 技术选型

8.1 推荐方案一：纯前端 + Node.js 后端（推荐）

层级	技术	说明
前端	HTML5 + CSS3 + Vanilla JS	轻量，无需框架构建步骤
前端增强	Bootstrap 5	快速构建响应式 UI
HTTP 请求	Fetch API / Axios	与后端 API 通信
后端	Node.js + Express.js	轻量 Web 框架
数据库驱动	mysql2 (Node.js)	连接 MySQL
数据库	MySQL 8.0	关系型数据库
开发工具	VS Code + MySQL Workbench	IDE + 数据库管理

8.2 推荐方案二：纯前端 + Python 后端

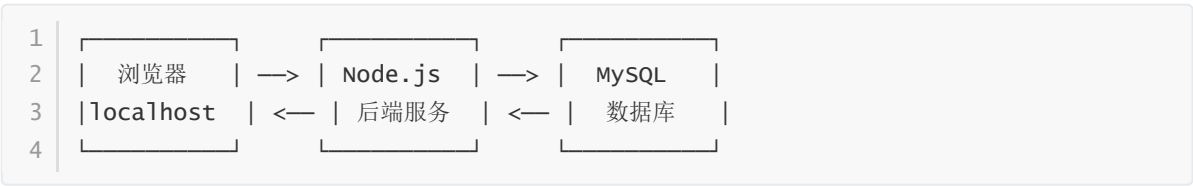
层级	技术	说明
后端	Python 3 + Flask	轻量 Python Web 框架
数据库驱动	PyMySQL / mysql-connector-python	Python 连接 MySQL
其他	同方案一	

8.3 图片存储方案

- 角色图片、声优照片等使用 **URL 存储**
- 图片可托管于免费图床（如 SM.MS、ImgURL）或本地 `/static/images/` 目录
- 数据库仅存储图片 URL 字符串

9. 部署与公开方案

9.1 本地运行（个人使用）



- **代价：**零成本，但仅本机可访问
- **适用：**个人使用、开发调试、课程演示

9.2 局域网共享

- 将后端服务绑定到 `0.0.0.0`，同局域网设备通过 IP 访问
- **代价：**零成本，但范围限于同一网络（如校园网）

9.3 公网部署

方案	月成本 (约)	说明
云服务器 (VPS)	¥50-100/月	阿里云/腾讯云轻量应用服务器，需自行配置环境
免费托管 + 云数据库	¥0-30/月	前端用 GitHub Pages/Vercel（免费），后端用 Railway/Render（有免费额度），数据库用 PlanetScale（免费 5GB）或 Aiven（免费）
学生优惠	¥0-10/月	阿里云/腾讯云有学生机优惠，约 ¥10/月

公网部署代价说明

1. **金钱成本：**最低可做到免费（使用 Vercel + PlanetScale 免费额度），但免费额度有限制（并发、存储等）
2. **时间成本：**需要配置域名 DNS、HTTPS 证书、环境变量、数据库迁移等
3. **安全成本：**公网暴露面临被攻击风险，需要做基本安全防护（SQL 注入防护、XSS 防护、CORS 配置等）
4. **维护成本：**需要定期更新依赖、备份数据库、监控服务状态

建议：课程演示阶段使用**本地运行**即可满足要求。如需展示给他人，可使用**局域网共享**或**Vercel 免费部署方案**。

10. 后期预期进展

10.1 第一阶段：数据库设计与初始化（第 1 周）

- ☒ 完成需求分析与架构设计（本文档）
- ☒ 设计 ER 图与数据库表结构
- ☐ 编写 SQL DDL 建表脚本
- ☐ 编写测试数据 INSERT 脚本（预置 LoveLive 系列数据）
- ☐ 在 MySQL 中创建数据库并验证表结构

10.2 第二阶段：后端 API 开发（第 2-3 周）

- ☐ 搭建 Node.js Express / Python Flask 项目骨架
- ☐ 实现角色介绍子系统 API:
 - GET /api/projects — 企划列表
 - GET /api/groups — 动漫团列表（支持筛选）
 - GET /api/groups/:id — 动漫团详情（含成员）
 - GET /api/characters — 角色列表（支持角色名、团名筛选）
 - GET /api/characters/:id — 角色详情
 - GET /api/seiyuus — 声优列表（支持年龄范围筛选）
 - GET /api/seiyuus/:id — 声优详情
- ☐ 实现排练子系统 API:
 - GET /api/dance-groups — 舞团列表
 - GET /api/dance-groups/:id — 舞团详情（含成员和排练）
 - POST /api/rehearsals — 新增排练（含人数上限）
 - GET /api/rehearsals — 查询排练（支持多维筛选，**返回含 current_participants 和 occupancy_rate**）
 - GET /api/rehearsals/:id — 排练详情（含参与角色、人数统计）
 - PUT /api/rehearsals/:id — 修改排练（可修改人数上限、增删参与人员）
 - DELETE /api/rehearsals/:id — 删除排练
 - POST /api/rehearsals/:id/participants — 添加参与记录（满员时返回 409 错误）
 - GET /api/rehearsals/stats/occupancy — **获取各时段人数统计**（按日期/舞团聚合饱和度数据，供前端图表使用）

10.3 第三阶段：前端页面开发（第 3-4 周）

- ☐ 搭建前端页面框架（导航栏、布局、路由）
- ☐ 实现角色介绍模块页面：
 - 企划列表 / 动漫团详情页
 - 角色列表 / 角色详情页
 - 声优列表 / 筛选功能
- ☐ 实现排练模块页面：

- 舞团列表 / 舞团详情页
 - 排练日历视图 (含**人数饱和度颜色标识和进度条**)
 - 排练详情中的人数面板 (当前人数/上限 + 参与舞见列表)
 - 排练新增/编辑模态框 (含**人数上限设置**)
 - 排练删除确认
 - **人数饱和度筛选器** (全部 / 有空位 / 将满 / 已满)
- ☐ **实现月视图人数热力图** (日期格内用颜色圆点标识各排练饱和度)
 - ☐ 实现跨系统跳转链接
 - ☐ 前端表单验证 (含满员时阻止添加的提示)

10.4 第四阶段：联调测试与优化 (第 5 周)

- ☐ 前后端联调, 修复接口问题
- ☐ 测试所有 CRUD 功能和筛选功能 (**重点测试人数统计准确性和满员拒绝逻辑**)
- ☐ 测试跨系统跳转链路
- ☐ **验证人数可视化效果**: 各饱和度颜色是否正确、进度条是否准确、月视图热力图是否可读
- ☐ UI/UX 细节优化
- ☐ 响应式适配验证
- ☐ 撰写用户使用说明

10.5 第五阶段：扩展功能 (可选)

- ☐ 排练冲突检测 (同一舞团同一时间段不能安排两场排练)
- ☐ **人数趋势图表** (使用 Chart.js 展示某舞团近期排练人数变化趋势)
- ☐ 数据统计仪表盘 (排练总次数、最常被扮演的角色、**平均参与人数、满员率**等)
- ☐ 排练提醒通知 (排练前一天提醒 + 人数不足提醒)
- ☐ 数据导出 (CSV/Excel)
- ☐ 用户登录与权限管理
- ☐ 暗色模式

附录 A: SQL 建表脚本 (预览)

```
1  -- 创建数据库
2  CREATE DATABASE IF NOT EXISTS lovelive_db
3      DEFAULT CHARACTER SET utf8mb4
4      DEFAULT COLLATE utf8mb4_unicode_ci;
5
6  USE lovelive_db;
7
8  -- 企划表
9  CREATE TABLE project (
10     project_id INT AUTO_INCREMENT PRIMARY KEY,
11     name       VARCHAR(100) NOT NULL UNIQUE,
12     description TEXT,
13     start_date DATE,
```

```

14     end_date    DATE
15 ) ENGINE=InnoDB;
16
17 -- 动漫团表
18 CREATE TABLE anime_group (
19     group_id      INT AUTO_INCREMENT PRIMARY KEY,
20     project_id    INT NOT NULL,
21     name          VARCHAR(100) NOT NULL,
22     description    TEXT,
23     representative_works TEXT,
24     history        TEXT,
25     founding_date  DATE,
26     disband_date   DATE,
27     image_url      VARCHAR(500),
28     FOREIGN KEY (project_id) REFERENCES project(project_id)
29         ON DELETE CASCADE ON UPDATE CASCADE,
30     INDEX idx_group_project (project_id),
31     INDEX idx_group_name (name)
32 ) ENGINE=InnoDB;
33
34 -- 声优表
35 CREATE TABLE seiyuu (
36     seiyuu_id INT AUTO_INCREMENT PRIMARY KEY,
37     name      VARCHAR(100) NOT NULL,
38     birth_date DATE,
39     gender     ENUM('女','男','其他') DEFAULT '女',
40     agency     VARCHAR(100),
41     image_url  VARCHAR(500),
42     INDEX idx_seiyuu_gender_birth (gender, birth_date)
43 ) ENGINE=InnoDB;
44
45 -- 角色表
46 CREATE TABLE `character` (
47     character_id INT AUTO_INCREMENT PRIMARY KEY,
48     group_id     INT NOT NULL,
49     seiyuu_id    INT,
50     name         VARCHAR(100) NOT NULL,
51     image_url    VARCHAR(500),
52     description   TEXT,
53     birthday     DATE,
54     blood_type   ENUM('A','B','AB','O','不明'),
55     height       DECIMAL(5,1),
56     hobby        VARCHAR(500),
57     FOREIGN KEY (group_id) REFERENCES anime_group(group_id)
58         ON DELETE CASCADE ON UPDATE CASCADE,
59     FOREIGN KEY (seiyuu_id) REFERENCES seiyuu(seiyuu_id)
60         ON DELETE SET NULL ON UPDATE CASCADE,
61     INDEX idx_char_group (group_id),
62     INDEX idx_char_name (name),
63     INDEX idx_char_seiyuu (seiyuu_id)
64 ) ENGINE=InnoDB;
65
66 -- 舞团表
67 CREATE TABLE dance_group (
68     dance_group_id INT AUTO_INCREMENT PRIMARY KEY,
69     anime_group_id INT,

```

```

70     name          VARCHAR(100) NOT NULL,
71     description    TEXT,
72     created_date   DATE,
73     FOREIGN KEY (anime_group_id) REFERENCES anime_group(group_id)
74         ON DELETE SET NULL ON UPDATE CASCADE,
75     INDEX idx_dg_anime (anime_group_id)
76 ) ENGINE=InnoDB;
77
78 -- 舞见表
79 CREATE TABLE dancer (
80     dancer_id      INT AUTO_INCREMENT PRIMARY KEY,
81     dance_group_id INT NOT NULL,
82     cn_name        VARCHAR(100) NOT NULL,
83     contact_info   VARCHAR(200),
84     FOREIGN KEY (dance_group_id) REFERENCES dance_group(dance_group_id)
85         ON DELETE CASCADE ON UPDATE CASCADE,
86     INDEX idx_dancer_group (dance_group_id)
87 ) ENGINE=InnoDB;
88
89 -- 排练表
90 CREATE TABLE rehearsal (
91     rehearsal_id   INT AUTO_INCREMENT PRIMARY KEY,
92     dance_group_id INT NOT NULL,
93     rehearsal_date DATE NOT NULL,
94     start_time     TIME NOT NULL,
95     end_time       TIME NOT NULL,
96     location       VARCHAR(200) NOT NULL,
97     max_participants INT DEFAULT 0 COMMENT '人数上限, 0表示不限制',
98     content_summary TEXT,
99     FOREIGN KEY (dance_group_id) REFERENCES dance_group(dance_group_id)
100         ON DELETE CASCADE ON UPDATE CASCADE,
101     INDEX idx_reh_date (rehearsal_date),
102     INDEX idx_reh_group (dance_group_id)
103 ) ENGINE=InnoDB;
104
105 -- 排练参与表
106 CREATE TABLE rehearsal_participation (
107     participation_id INT AUTO_INCREMENT PRIMARY KEY,
108     rehearsal_id     INT NOT NULL,
109     dancer_id        INT NOT NULL,
110     character_id     INT NOT NULL,
111     FOREIGN KEY (rehearsal_id) REFERENCES rehearsal(rehearsal_id)
112         ON DELETE CASCADE ON UPDATE CASCADE,
113     FOREIGN KEY (dancer_id) REFERENCES dancer(dancer_id)
114         ON DELETE CASCADE ON UPDATE CASCADE,
115     FOREIGN KEY (character_id) REFERENCES `character`(character_id)
116         ON DELETE CASCADE ON UPDATE CASCADE,
117     UNIQUE KEY uk_r_d_c (rehearsal_id, dancer_id, character_id),
118     INDEX idx_rp_rehearsal (rehearsal_id),
119     INDEX idx_rp_dancer (dancer_id),
120     INDEX idx_rp_character (character_id)
121 ) ENGINE=InnoDB;
122
123 -- =====
124 -- 视图: 排练人数统计 (含饱和度)
125 -- =====

```

```
126 CREATE VIEW rehearsal_with_count AS
127 SELECT
128     r.rehearsal_id,
129     r.dance_group_id,
130     r.rehearsal_date,
131     r.start_time,
132     r.end_time,
133     r.location,
134     r.max_participants,
135     r.content_summary,
136     COUNT(rp.participation_id) AS current_participants,
137     CASE
138         WHEN r.max_participants = 0 THEN NULL
139         ELSE ROUND(COUNT(rp.participation_id) / r.max_participants * 100,
140 1)
141     END AS occupancy_rate,
142     CASE
143         WHEN r.max_participants = 0 THEN 'unlimited'
144         WHEN COUNT(rp.participation_id) >= r.max_participants THEN 'full'
145         WHEN COUNT(rp.participation_id) / r.max_participants >= 0.8 THEN
146 'near_full'
147         WHEN COUNT(rp.participation_id) / r.max_participants >= 0.5 THEN
148 'half'
149         ELSE 'available'
150     END AS occupancy_status
151 FROM rehearsal r
152 LEFT JOIN rehearsal_participation rp ON r.rehearsal_id = rp.rehearsal_id
153 GROUP BY r.rehearsal_id;
```

附录 B：图例说明

符号	含义
PK	Primary Key（主键）
FK	Foreign Key（外键）
<code> --o{</code>	一对多关系（Mermaid 语法）
<code>ON DELETE CASCADE</code>	删除父记录时级联删除子记录
<code>ON DELETE SET NULL</code>	删除父记录时将外键设为 NULL

